Yes. Based on everything we've worked through over the last few weeks (latest MoMs, scope changes, D2/D3/D4 deliverables, HMI architecture, Process Twin architecture, RXU architecture, and your latest architecture diagrams), I think WP5.3 has reached the point where it deserves a **proper Software Architecture Document (SAD)** rather than just a list of deliverables.

I would recommend writing it using **arc42**, because:

- Airbus architecture teams are familiar with architecture documentation.
- It separates business goals from technical implementation.
- It can evolve as development progresses.
- It aligns well with HLD and later LLD.

Proposed arc42 Structure for WP5.3 Process Optimization

This document should **not** describe only the HMI.

Instead it should describe the complete **Process Twin Platform**.

WP5.3 Process Optimization
Software Architecture Document

Version 1.0

---

1. Introduction & Goals

Purpose

Describe the architecture of WP5.3 Process Optimization platform.

The platform performs

- process data ingestion
- engineering data preparation
- process analytics
- time-series analysis

- system identification
- process anomaly analysis
- HMI visualization

using production data acquired from WP5.1.

Business Goals

• Improve process understanding

• Detect process anomalies

• Learn expected process behaviour

• Compare expected vs actual behaviour

• Provide engineering visualization

• Support process optimization

• Support engineering investigation

Quality Goals

Performance

Scalability

Maintainability

Extensibility

Machine independence

Reusable analytics

Near real-time visualization

---

2. Constraints

Examples

No physics-based digital twin

No robot simulation

No CAD behaviour simulation

No online optimization

Data comes from WP5.1

Historical data

Live data

Edge deployment

IPC deployment

HMI developed using RXU framework

---

3. Context

System Context Diagram

Here include your High-Level HMI Architecture.

The diagram should show

DAQ
↓

WP5.1

↓

Database

↓

WP5.3

↓

HMI

↓

Operator

External systems

RXU Connectivity

RXU Data

Overall Orchestrator

DAQ

Database

Operator

---

4. Solution Strategy

Describe why architecture is divided into modules.

For example

Process Analytics

↓

Time-Series

↓

Behaviour Model

↓

Visualization

instead of one monolithic application.

Explain

Loose coupling

Independent deployment

Machine independence

Reusable services

Microservice architecture

Topic-based communication

---

5. Building Block View

This becomes the biggest chapter.

Level 1

Whole system

Exactly your detailed architecture.

Frontend

↓

Backend

↓

Analytics Engine

↓

Database

↓

DAQ

↓

Connectivity

Level 2

Backend

Break backend into

API Gateway

↓

Authentication

↓

Visualization Service

↓

Analytics Service

↓

Geometry Service

↓

Storage

↓

Logging

↓

Configuration

↓

Event Manager

↓

Streaming Manager

Level 2

Analytics

Break into

Data Ingestion

↓

Data Preparation

↓

Engineering Analytics

↓

Time-Series

↓

Behaviour Model

↓

Root Cause

↓

Report Generator

Level 2

Frontend

Modules

Dashboard

KPI Trends

3D Viewer

Time-Series

Historical Replay

Root Cause View

Settings

User Management

Notification

---

6. Runtime View

Very important.

Sequence diagrams.

Example

Historical replay

Operator

↓

Frontend

↓

Backend

↓

Database

↓

Data Analysis

↓

Response

↓

Visualization

Live data

DAQ

↓

RXU Connectivity

↓

Backend

↓

Analytics

↓

Frontend

Process anomaly

DAQ

↓

Time-Series

↓

Anomaly

↓

Backend

↓

Frontend

↓

Operator

Expected vs Actual

Behaviour Model

↓

Prediction

↓

Compare

↓

Deviation

↓

Visualization

---

7. Deployment View

IPC

Industrial PC

Ubuntu

↓

Docker

↓

Frontend

↓

Backend

↓

Analytics

↓

Time-Series

↓

System Identification

↓

Database

Connections

Database

DAQ

PLC

Robot

HMI

---

8. Cross Cutting Concepts

Logging

Configuration

Database

Streaming

Caching

Authentication

Visualization

Error Handling

Health Monitoring

Versioning

---

9. Architectural Decisions

ADR list.

Examples

Why REST

Why WebSockets

Why RXU

Why Time-Series separated

Why Behaviour model separated

Why Edge deployment

Why IPC

Why API Gateway

10. Quality Requirements

Performance

Latency

Availability

Maintainability

Scalability

Security

Robustness

Testability

---

11. Risks

Incomplete datasets

Poor timestamps

DAQ latency

Synchronization

Large CAD files

High visualization load

Scalability

Model accuracy

---

12. Glossary

DAQ

KPI

IPC

RXU

HMI

System Identification

Behaviour Model

Time-Series

Expected Behaviour

Process Anomaly

Root Cause

Additional Appendix (Highly Recommended)

Since this project is R&T, I strongly recommend adding appendices that go beyond the standard arc42 template:

Appendix A – Process Data Flow

DAQ

↓

Data Preparation

↓

Engineering Analytics

↓

Time-Series

↓

System Identification

↓

Visualization

↓

Operator

Appendix B – Analytics Pipeline

Historical Data

↓

Cleaning

↓

Feature Engineering

↓

Visualization Dataset

↓

Time-Series Dataset

↓

Behaviour Dataset

Appendix C – HMI Module Description

Describe each HMI module:

- Dashboard
- KPI Trends
- 3D Geometry Viewer
- Historical Replay
- Process Timeline
- Expected vs Actual
- Root Cause Analysis
- Alarm Panel
- Settings
- User Management

Appendix D – API Catalog

List every backend API:

- KPI API
- Geometry API
- Historical Replay API
- Time-Series API
- Root Cause API
- Configuration API
- Authentication API
- Logging API

Appendix E – Data Model

Document the schemas for:

- KPI
- Time Series
- Geometry Mapping
- Process State
- Root Cause
- Event
- Metadata

My recommendation

Given the maturity of your WP5.3 definition, I would not stop at a simple arc42 document. I would prepare a **100–150 page architecture specification** that combines:

1. **arc42 Software Architecture Document (SAD)** – overall architecture and design.
2. **Software Design Description (SDD)** – module-level design, APIs, class diagrams, deployment, and sequence diagrams.
3. **Interface Control Document (ICD)** – communication between WP5.1, WP5.2, WP5.3, RXU Connectivity, the database, and the HMI.
4. **Data Model Specification** – schemas for time-series, KPIs, geometry mappings, events, and metadata.
5. **API Specification** – REST/WebSocket endpoints and message contracts.

This would provide a complete architectural baseline for WP5.3, from high-level design down to implementation details, while remaining aligned with the scope you've defined.

can you give detailed documentation about different HMI back end componnets i am expecting detailed docuemntattion about high level and detailed architecture what each compoenents does, communication between compoenents etc

WP5.3 Process Twin HMI Backend

High-Level and Detailed Architecture Documentation

1. Purpose

The HMI backend provides the software layer between:

- the HMI frontend,
- WP5.1 data storage,
- live machine-data connectivity,
- WP5.3 process-analysis modules,
- and the RXU orchestration framework.

Its main responsibilities are to:

- retrieve historical and NC-program data,
- receive live process data,
- coordinate data-processing and analysis services,
- prepare visualization-ready results,
- expose results to the frontend,
- manage execution status, errors and logs.

The backend is not responsible for acquiring raw sensor signals directly from hardware when WP5.1 or RXU Connectivity already performs that function.

The intended flow is:

Machine / DAQ / WP5.1
↓
RXU.Data and RXU Connectivity
↓
WP5.3 Processing and Analysis Services
↓
Backend Interface / API Gateway
↓
HMI Frontend

---

2. Scope Boundary

Included in the backend

- Frontend API interface
- Historical-data access
- Live-data subscription
- Data-ingestion coordination
- Visualization-data processing
- TCP and NC-path analysis
- Time-series anomaly-analysis integration
- System-identification output integration
- Root-cause investigation-data integration
- Execution orchestration
- Status and progress reporting
- Logging and diagnostics

Outside the backend scope

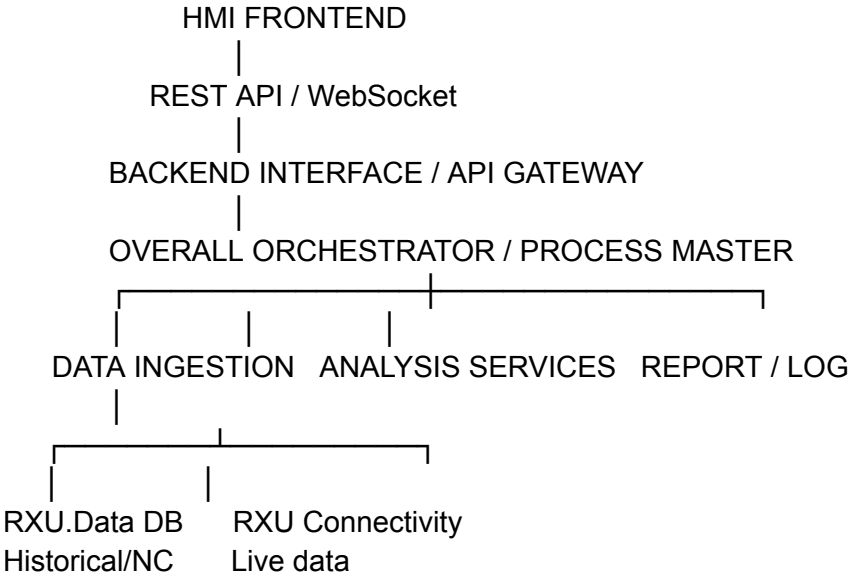
- Raw DAQ hardware control
- Primary sensor filtering owned by WP5.1
- Machine or robot motion control
- Closed-loop parameter adjustment
- Virtual commissioning
- Physics-based robot or aircraft simulation
- Automated defect prediction

- Automatic production optimization

---

3. High-Level Architecture

Your high-level architecture contains five principal backend-facing components.



---

4. High-Level Component Descriptions

4.1 HMI Frontend

The frontend is the user-facing web application hosted on the IPC or accessed through a browser connected to the IPC.

It is responsible for:

- displaying KPI dashboards,
- rendering charts,
- showing TCP trajectories,
- loading 3D geometry,
- displaying planned and actual paths,
- showing process anomalies,
- presenting root-cause investigation information,

- allowing the user to select runs, time windows and KPIs.

The frontend should not directly query databases, RXU Connectivity or analysis modules. All access should go through the backend interface.

Communication

Frontend → Backend:
REST request or WebSocket subscription

Backend → Frontend:
JSON response, live update, job status or error

4.2 Backend Interface / API Gateway

The Backend Interface is the single controlled entry point between the frontend and all backend services.

It hides the internal RXU service structure from the frontend.

Its responsibilities include:

- receiving frontend requests,
- validating request parameters,
- selecting the correct backend workflow,
- starting an analysis job through the orchestrator,
- retrieving results,
- returning normalized response payloads,
- streaming live updates through WebSockets,
- returning job progress and error status.

Example requests

GET /api/runs/{runId}/kpis
GET /api/runs/{runId}/trajectory
GET /api/runs/{runId}/geometry-overlays
POST /api/analysis/path-deviation
POST /api/analysis/root-cause
WS /api/live/kpis

Important design principle

The API Gateway should not contain heavy analysis logic. It should only:

Validate → Route → Track → Return

4.3 Overall Orchestrator / Process Master

The orchestrator coordinates multi-step workflows involving several RXU applications.

It may be implemented using:

- a behaviour tree,
- a workflow engine,
- a state machine,
- or RXU service orchestration.

It is responsible for:

- interpreting requests received from the backend interface,
- selecting required services,
- sequencing execution,
- tracking dependencies,
- monitoring service status,
- handling retries and failures,
- collecting results,
- notifying the frontend when processing is complete.

Example workflow

For a request to visualize actual TCP path against the NC path:

1. Receive request from API Gateway
2. Fetch run metadata
3. Request historical TCP data
4. Request NC-program path
5. Invoke geometry/path mapping service
6. Invoke path-deviation analysis
7. Store or cache the result
8. Return visualization payload to API Gateway

Why an orchestrator is needed

Without orchestration, the frontend or API layer would need to know:

- which services to call,
- in what sequence,
- how to handle partial failures,
- and how to combine results.

The orchestrator centralizes that logic.

---

5. Detailed Backend Architecture

5.1 Data Ingestion Module

Purpose

The Data Ingestion module provides a common interface for historical and live process data.

It should ingest data from two primary sources:

- **RXU.Data / WP5.1 database** for historical, validated and NC-program data.
- **RXU Connectivity** for live machine and process data.

Inputs

Historical input may include:

- KPI values,
- machine states,
- TCP position and orientation,
- event logs,
- process-state information,
- run and part metadata,
- NC-programmed path,
- course and layer details.

Live input may include:

- current KPI values,
- current machine state,
- current TCP pose,
- alarms and events,
- run progress.

Internal subcomponents

Historical Data Adapter

Queries RXU.Data and converts database records into the internal WP5.3 data format.

Live Data Subscriber

Subscribes to RXU topics published by RXU Connectivity.

Data Source Registry

Maintains configuration describing:

- source names,
- topics,
- database tables,
- data types,
- units,
- and update frequencies.

Buffer Manager

Temporarily stores incoming live data until it is consumed or persisted.

Run Context Resolver

Associates every record with:

- run ID,
- part ID,
- layer,
- course,
- process state,
- and timestamp.

Outputs

The ingestion module produces normalized internal messages such as:

```
{
  "runId": "RUN-2026-001",
  "timestamp": "2026-07-13T10:15:32.125Z",
  "signal": "tow_tension",
  "value": 21.4,
  "unit": "N",
  "processState": "laying",
  "source": "RXU.Connectivity"
}
```

Communication

RXU.Data → Data Ingestion:

Database query / repository interface

RXU Connectivity → Data Ingestion:
RXU topic subscription

Data Ingestion → Orchestrator:
RXU service response or normalized data topic

5.2 Data Preparation Module

Purpose

The Data Preparation module converts the already cleaned and validated WP5.1 data into structures required by WP5.3 services.

It must not duplicate WP5.1 responsibilities such as primary filtering and raw-signal validation.

Responsibilities

- align data to a common run timeline,
- associate data with process states,
- align TCP data with KPI timestamps,
- map NC-path records to actual execution records,
- convert units where required,
- enrich data with part, run, layer and course context,
- generate analysis-ready and visualization-ready structures.

Typical processing

Time alignment

Associates KPI values, TCP pose and machine events using timestamps.

Process-state tagging

Marks each time interval as:

- laying,
- travel,
- pause,
- transition,
- alarm,
- or other available state.

Spatial association

Links:

Timestamp → TCP pose → geometry location

Dataset slicing

Extracts selected:

- run,
- layer,
- course,
- time range,
- or process state.

Output

- synchronized run dataset,
- TCP trajectory dataset,
- KPI trend dataset,
- planned-path dataset,
- actual-path dataset,
- geometry-mapping dataset.

5.3 Process Data Analysis Module

Purpose

This module performs engineering data analysis required for HMI visualization before advanced time-series anomaly analysis and system identification are available.

It should remain separate from anomaly-detection and behavioral-model modules.

Responsibilities

- calculate KPI statistics,
- calculate path and velocity deviation,
- generate planned-versus-actual comparison,
- create TCP-to-geometry mappings,
- prepare KPI overlays,
- create run and process summaries.

Internal submodules

KPI Calculation Service

Calculates:

- minimum,
- maximum,
- average,
- standard deviation,
- moving average,
- change rate,
- run-level summaries.

TCP Trajectory Service

Creates ordered TCP path data from time-stamped poses.

Planned-vs-Actual Analysis

Compares actual execution against NC-programmed data.

Potential outputs include:

- positional deviation,
- path deviation,
- speed difference,
- execution-timing difference.

Geometry Mapping Service

Maps TCP points and KPI values to the part representation.

Visualization Payload Builder

Converts analysis results into structures suitable for:

- charts,
- heatmaps,
- paths,
- markers,
- tooltips,
- and timeline views.

Example output

{

```
  "runId": "RUN-2026-001",
  "overlayType": "kpi_path",
  "kpi": "tow_tension",
  "points": [
    {
      "x": 1.24,
      "y": 0.42,
      "z": 2.11,
      "value": 21.4,
      "timestamp": "2026-07-13T10:15:32.125Z"
    }
  ]
}
```

5.4 Time-Series Analysis Service

Purpose

The Time-Series Analysis service detects abnormal temporal process behaviour.

It is distinct from basic data analysis.

Basic data analysis prepares and visualizes process data. Time-series analysis determines whether a signal exhibits abnormal behaviour.

Functions

- threshold evaluation,
- spike detection,
- drift detection,
- oscillation detection,
- process-state-aware anomaly detection,
- anomaly scoring,
- severity classification.

Inputs

- synchronized KPI sequences,
- process states,
- configured limits,
- window definitions,
- historical reference data.

Outputs

Anomaly type
Start timestamp
End timestamp
Related KPI
Severity
Score
Process state
Associated TCP region

Communication

Orchestrator → Time-Series Service:
Run ID, selected signals, configuration

Time-Series Service → Orchestrator:
Anomaly result set

Orchestrator → Backend Interface:
Visualization-ready anomaly response

5.5 System Identification / Behavioural Model Service

Purpose

The System Identification service learns or applies the expected process behaviour under given operating conditions.

It is not a 3D digital twin and does not simulate the robot or aircraft environment.

Functions

- input-output mapping,
- expected process behaviour estimation,
- model parameter estimation,
- process-state-aware behaviour modelling,
- expected-versus-actual comparison,
- residual calculation,
- process-envelope generation.

Example model logic

Inputs:

Speed, temperature, process state, pressure

Model output:
Expected tension behaviour

Comparison:
Expected tension vs actual tension

Outputs

- expected KPI values,
- expected process trends,
- nominal operating envelopes,
- residuals,
- deviation indicators,
- model-health metadata.

Communication

The service can be called by the orchestrator for:

- a selected historical run,
- a selected time interval,
- or a configured live-evaluation window if later required.

5.6 Root-Cause Analysis / Investigation Service

Purpose

The Root-Cause Analysis service should support engineering investigation of process anomalies.

Unless validated domain models exist, it should not claim to provide automatic or definitive root-cause diagnosis.

Functions

- correlate anomaly timestamp with other process variables,
- retrieve the time window before and after the anomaly,
- identify related events,
- identify coincident KPI variations,
- associate anomaly with TCP and geometry region,

- rank possible contributing variables,
- prepare investigation context for the HMI.

Inputs

- anomaly event,
- synchronized KPI data,
- process-state timeline,
- TCP path,
- machine events,
- expected-versus-actual outputs.

Outputs


```
{
  "anomalyId": "ANOM-105",
  "primarySignal": "tow_tension",
  "contextWindow": {
    "beforeSeconds": 10,
    "afterSeconds": 10
  },
  "relatedSignals": [
    {
      "name": "laying_speed",
      "relationship": "increase_before_event"
    }
  ],
  "relatedEvents": [
    {
      "type": "process_state_transition",
      "timestamp": "..."
    }
  ]
}
```


Recommended HMI wording

Use:

- “Possible contributing variables”
- “Associated events”
- “Investigation context”

Avoid:

- “Confirmed root cause”
- “Automatic diagnosis”

unless this capability is formally validated.

5.7 Report and Log Generation Module

Purpose

This module creates technical logs and structured output records for traceability.

Responsibilities

- record analysis requests,
- log workflow execution,
- capture service errors,
- store analysis configuration,
- generate run summary data,
- generate anomaly summary data,
- create downloadable reports only if required.

Log categories

- system log,
- data-ingestion log,
- orchestration log,
- API-access log,
- analysis-execution log,
- error log,
- audit log.

5.8 RXU.Data Interface

Purpose

RXU.Data acts as the historical-data repository supplied by WP5.1.

It may contain:

- process data,

- machine data,
- TCP data,
- NC data,
- part and run metadata,
- events,
- analysis outputs if results are persisted.

Access pattern

The backend should not allow arbitrary direct database access from the frontend.

Recommended flow:

Frontend
↓
API Gateway
↓
Orchestrator / Data Service
↓
RXU.Data

5.9 RXU Connectivity

Purpose

RXU Connectivity provides the bridge to live machine, robot and DAQ information.

It publishes selected process data through RXU topics or services.

Responsibilities

- machine/robot communication,
- protocol abstraction,
- connection monitoring,
- live-data publication,
- reconnection and communication-health handling.

Examples of live topics

/process/kpi/tension
/process/kpi/speed
/machine/state
/robot/tcp_pose
/process/event

Scope consideration

WP5.3 consumes this information. It should not reproduce the device-driver or machine-protocol logic already implemented in RXU Connectivity.

---

6. Communication Patterns

6.1 Frontend to Backend

REST

Used for:

- fetching run lists,
- loading historical KPI data,
- loading geometry,
- submitting analysis requests,
- retrieving completed results.

WebSocket

Used for:

- live KPI updates,
- job-progress updates,
- live event or anomaly notifications,
- connectivity and service-health updates.

6.2 Backend to RXU Services

RXU topics are suitable for:

- asynchronous live data,
- event publication,
- status changes.

RXU services are suitable for:

- request-response workflows,
- retrieving historical data,
- starting analysis,
- requesting results.

6.3 Backend to Database

A repository or data-access service should isolate database-specific details.

Backend Service
↓
Repository Interface
↓
RXU.Data

This avoids coupling business logic directly to database tables.

---

7. Runtime Communication Scenarios

7.1 Historical KPI Visualization

1. User selects run in frontend.
2. Frontend calls GET /runs/{id}/kpis.
3. API Gateway validates the request.
4. Orchestrator requests historical data.
5. Data Ingestion retrieves data from RXU.Data.
6. Data Preparation aligns and structures it.
7. Process Data Analysis calculates summaries.
8. Payload Builder creates chart-ready JSON.
9. API Gateway returns the response.
10. Frontend renders cards and trend charts.

7.2 Live KPI Visualization

1. RXU Connectivity publishes live KPI topic.
2. Data Ingestion subscribes to the topic.
3. Data is associated with the current run.

4. Backend publishes the update through WebSocket.
5. Frontend state is updated.
6. The visible chart and KPI card refresh.

7.3 TCP and KPI Overlay on Geometry

1. Frontend requests a geometry overlay.
2. API Gateway sends request to orchestrator.
3. Orchestrator retrieves:
   - geometry reference,
   - TCP trajectory,
   - selected KPI data.
4. Geometry Mapping Service associates KPI values with TCP points.
5. Visualization Payload Builder creates the overlay payload.
6. Frontend renders the path or heatmap on the 3D geometry.

7.4 Process-Anomaly Investigation

1. Time-Series Service detects an anomaly.
2. The anomaly result is returned to the orchestrator.
3. Root-Cause Investigation Service retrieves related signals and events.
4. Geometry Mapping Service resolves the affected location.
5. Backend returns:
   - anomaly details,
   - related KPI trends,
   - event context,
   - geometry position.
6. Frontend opens the anomaly-investigation view.

7.5 Expected-versus-Actual Analysis

1. Frontend requests behavioural analysis for a run.
2. Orchestrator retrieves the prepared model dataset.
3. System Identification service calculates expected behaviour.
4. Actual data is aligned with expected values.
5. Residuals and deviation indicators are calculated.
6. Backend returns trend and envelope payloads.
7. Frontend shows expected, actual and deviation views.

---

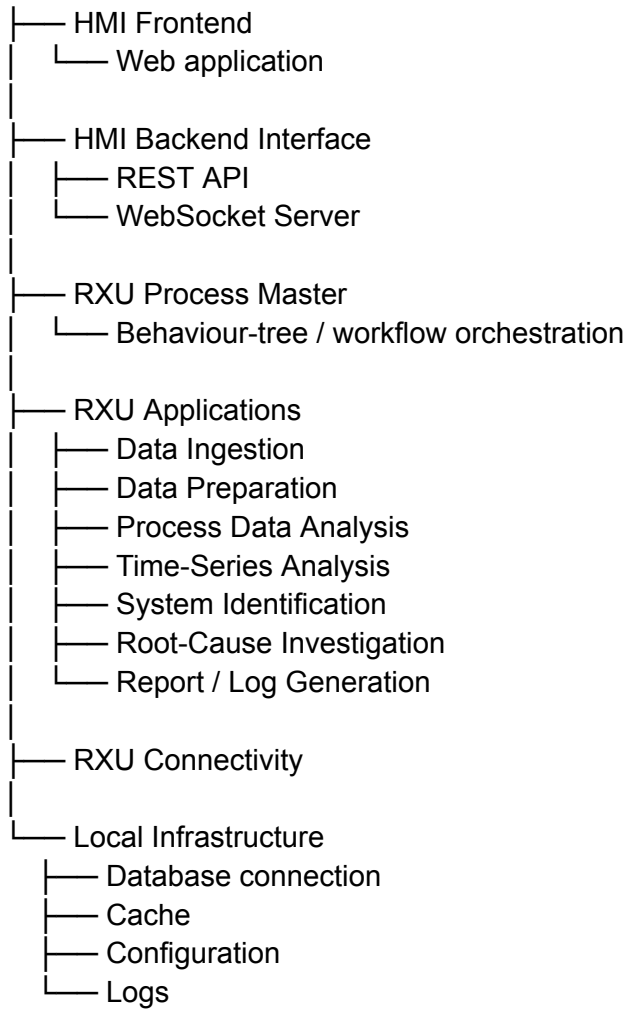
8. Backend Interface Matrix

Source	Consumer	Communication	Information
Frontend	API Gateway	REST/HTTPS	Queries and analysis requests
Frontend	API Gateway	WebSocket/WSS	Live subscription and updates
API Gateway	Orchestrator	RXU service/internal API	Workflow request
Orchestrator	Data Ingestion	RXU service/topic	Historical/live-data request
RXU.Data	Data Ingestion	Database/repository API	Historical and NC data
RXU Connectivity	Data Ingestion	RXU topic	Live process data
Orchestrator	Process Data Analysis	RXU service	KPI, TCP and geometry processing
Orchestrator	Time-Series Service	RXU service	Anomaly-analysis request
Orchestrator	System Identification	RXU service	Expected-behaviour evaluation
Orchestrator	Root-Cause Service	RXU service	Investigation request
Backend Services	Report/Log Module	Internal event/log API	Execution and diagnostic logs
API Gateway	Frontend	JSON/WebSocket	Visualization-ready results

---

9. Recommended Backend Packaging on the IPC

IPC / Edge Computer
|



Each service may be deployed as:

- an independent executable,
- a container,
- or an RXU application,

depending on the established RXU deployment standard.

---

10. Cross-Cutting Requirements

Configuration

Configuration should include:

- topic names,
- database connection,
- KPI definitions,
- units,
- service endpoints,
- geometry references,
- analysis parameters.

Error handling

Each service should return:

- status,
- error code,
- message,
- source service,
- request or job ID.

Health monitoring

Expose health for:

- API Gateway,
- orchestrator,
- database connection,
- RXU Connectivity,
- each analysis service.

Traceability

Every analysis request should contain a unique job ID so its execution can be followed across services.

Security

Where required:

- HTTPS/WSS,
- authenticated API access,
- user roles,
- audit logging,
- network separation between OT and IT.

Versioning

Version independently:

- API contracts,
- RXU messages,
- data schemas,
- analysis models,
- HMI frontend.

---

11. Important Architecture Clarification

The backend contains three fundamentally different processing categories:

Category	Purpose
Process Data Analysis	Prepare KPIs, TCP paths, geometry mapping and visual information
Time-Series Analysis	Detect spikes, drift, oscillations and abnormal process behaviour
System Identification	Estimate expected process behaviour and compare it against actual behaviour

Root-cause investigation consumes information from all three but should remain a separate service.

This separation allows the HMI to release basic visualization capability before the time-series and system-identification modules are fully ready.